

Dobot Magician Robot System: ROS 2, OpenCV, pydobot2 & PyQt Full Guide

Autonomous Pick-and-Place, Multi-Color Segmentation, 3-Phase Mission Execution & Collision Avoidance
(From Scratch)

Author / Architect: Thanadech (Google Deepmind Pair-Programming Assistant)

Operating System: Ubuntu Linux 22.04 LTS / ROS 2 Humble

Release Date: September 2026

📌 Guide Objective

This comprehensive documentation is tailored for students, engineers, and researchers starting with zero robotics or ROS 2 background. It explains every concept from the ground up: communication middleware, perspective computer vision, robot kinematics, asynchronous GUI design, 3-phase automated obstacle clearance, and goal collision avoidance.

Table of Contents

- **Chapter 1:** System Overview & Physical Field Geometry
- **Chapter 2:** ROS 2 Fundamentals from Scratch (Nodes, Topics, Messages)
- **Chapter 3:** Computer Vision Pipeline with OpenCV (V4L2, HSV, Morphology, Homography)
- **Chapter 4:** Dobot Magician Control with pydobot2 (UART, Cartesian PTP, End-Effectors)
- **Chapter 5:** GUI Architecture with PyQt (Event Loop, Signals & Slots, Thread-Safe RosBridge)
- **Chapter 6:** 3-Phase Mission Execution & Goal Collision Avoidance
- **Chapter 7:** UI Dashboard Walkthrough with Screenshots
- **Chapter 8:** Build, Execution & Troubleshooting Guide

Chapter 1: System Overview & Physical Field Geometry

1.1 Physical Layout & Coordinate Frame

The operational setup is configured on an international standard A4 sheet (210 x 297 mm) with four core spatial landmarks:

1. **Dobot Magician Robot Base:** Mounted at field origin coordinates ($X = 105.0$ mm, $Y = 270.0$ mm).
2. **3x3 Pallet Grid:** An outer bounding footprint of 114.5 x 114.5 mm with a 35.0 mm cell pitch. The 8 outer grid cells host colored cubes (Red, Yellow, Green, Blue, Orange, Purple, Cyan).
3. **Center Goal Cell [1, 1]:** The central drop target where selected cubes are vertically stacked (strictly locked to a 4-block maximum ceiling).
4. **4 Feeder Slots (Feeder 1 to 4):** Situated on the left perimeter strip ($Y \sim +79.3$ mm). Marked with green rings, these act as designated holding stations for obstacle cubes.

```
[4 Feeder Slots]      [3x3 Pallet Grid: 114.5 x 114.5 mm]
(Green Rims)
( #1 )                +-----+-----+-----+
                        | (0,0) | (0,1) | (0,2) |
                        +-----+-----+-----+
( #2 )                | (1,0) | (1,1) | (1,2) |
                        |          GOAL          |
                        +-----+-----+-----+
( #3 )                | (2,0) | (2,1) | (2,2) |
                        +-----+-----+-----+
( #4 )

                        [Dobot Magician Robot Base]
                        (105, 270) mm
```

1.2 Workspace Architecture (dobot_ws)

The workspace separates concerns into modular ROS 2 packages:

- `dobot_v2` : Core vision processing node, perspective transform math, hardware serial driver, and 3-phase trajectory sequencer.
- `dobot_ui` : Rich interactive PyQt dashboard providing camera streaming, live robot status telemetry, mission sequencing, and manual Cartesian jogging.

Chapter 2: ROS 2 Fundamentals from Scratch

2.1 What is ROS 2?

ROS 2 (Robot Operating System 2) is not an OS like Windows or Ubuntu; it is a distributed robotics software middleware. Built on the industrial **DDS (Data Distribution Service)** standard, ROS 2 enables separate programs (processes) to exchange data, synchronize clocks, and coordinate complex robotic tasks across multiple threads, processes, and network nodes seamlessly.

2.2 Core ROS 2 Primitives

1. Nodes (`rclpy.node.Node`)

A Node is an independent executable process responsible for a single task:

- `detection_node` : Captures webcam frames, segments cubes, and publishes detected metric coordinates.
- `dobot_controller` : Communicates over serial with the Dobot arm, manages safety limits, and executes trajectories.
- `dobot_ui_node` : Renders the GUI interface and transmits operator commands.

2. Topics & Publisher-Subscriber Pattern

Topics provide unidirectional, asynchronous streaming data channels. A Publisher broadcasts messages without knowing who is listening, and Subscribers trigger callback functions whenever a new message arrives.

Topic Name	Message Type	Publisher	Subscriber	Description
/detected_objects	std_msgs/msg/String (JSON)	detection_node	Dobot_UI , dobot_controller	Real-time metric coordinates (X, Y, Z), colors, and cell assignments.
/detected_objects_image/compressed	sensor_msgs/msg/CompressedImage	detection_node	Dobot_UI	Low-latency compressed JPEG stream rendered in the GUI viewer.
/dobot_status	std_msgs/msg/String (JSON)	dobot_controller	Dobot_UI	State telemetry: X, Y, Z, R Cartesian pose, suction status, connection state.
/dobot_ui_cmd	std_msgs/msg/String (JSON)	Dobot_UI	dobot_controller	Control messages: mission dispatch, manual jog, homing, emergency stop.

Chapter 3: Computer Vision Pipeline with OpenCV

3.1 Camera Capture & V4L2 Video Streaming

On Linux, USB webcams interface via the Video4Linux2 (`cv2.CAP_V4L2`) kernel driver. Using hardware MJPG decoding enables low-latency streaming at 640 x 480 resolution and 30 FPS without taxing the CPU:

```
import cv2

cap = cv2.VideoCapture('/dev/video0', cv2.CAP_V4L2)
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
ret, frame = cap.read() # Output is a BGR NumPy array
```

3.2 Why HSV is Superior to RGB/BGR for Color Segmentation

Standard cameras output images in **BGR (Blue, Green, Red)**. In BGR, illumination changes (shadows, daylight shifts, overhead flickering) simultaneously alter all three color channels, making thresholding unstable.

In contrast, **HSV (Hue, Saturation, Value)** isolates chromaticity from illumination:

- **H (Hue [0..180])**: Encodes the pure spectral color independent of light intensity (e.g. Green is always $\sim 35..85$ regardless of room lighting).
- **S (Saturation [0..255])**: Quantifies color purity, distinguishing vivid cube colors from white paper and highlights.
- **V (Value [0..255])**: Quantifies luminance, allowing dark shadows to be filtered out cleanly.

3.3 Color Masking & Morphological Noise Filtering

We apply `cv2.inRange()` to produce binary masks, followed by morphological opening to eliminate salt-and-pepper noise:

```
# Convert BGR to HSV
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

# Define HSV boundaries for Green
lower_green = np.array([35, 80, 70])
upper_green = np.array([85, 255, 255])
mask = cv2.inRange(hsv, lower_green, upper_green)

# Morphological opening (Erosion followed by Dilation) removes noise
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
mask_clean = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
```

3.4 Metric Perspective Homography

Because the overhead camera views the table at an oblique angle, perspective keystoneing distorts distances. We compute a 3x3 Homography transform matrix:

1. Detect the 4 outer corner coordinates of the grid in image pixel space (u, v).
2. Define target metric dimensions: a square 114.5 x 114.5 mm planar grid.
3. Calculate the matrix: `cv2.getPerspectiveTransform(src_pts, dst_pts)`.
4. Multiply any detected pixel centroid (u, v) by this homography matrix to obtain table-space millimeters (gx, gy), and map directly into Dobot robot base coordinates (rx, ry, rz)!

Chapter 4: Dobot Magician Control with pydobot2

4.1 Serial UART Protocol

The Dobot Magician communicates via a USB-to-UART serial interface at **115200 bps**. The protocol transmits structured binary packets: `Header (0xAA 0xAA) | Length | Command ID | Payload | Checksum`. The `pydobot2` library encapsulates this protocol into thread-safe Python methods.

4.2 Cartesian Workspace Limits

Dobot Magician Working Envelope

- **X (mm):** Forward/backward extension. Safe limits: 100.0 mm to 330.0 mm.
- **Y (mm):** Lateral left/right rotation. Limits: -250.0 mm to +250.0 mm.
- **Z (mm):** Vertical elevation. Limits: -60.0 mm to +160.0 mm.
- **R (deg):** End-effector wrist orientation. Limits: -180.0° to +180.0°.

4.3 PTP Motion Modes: MOVJ vs MOVL

- **MOVJ_XYZ (Joint Movement):** All joints interpolate simultaneously. The arm swings in a smooth, high-speed arc. Used for open transit between field waypoints.
- **MOVL_XYZ (Linear Movement):** The end-effector moves strictly along a Cartesian straight line. Crucial for picking up cubes vertically (downward 90°) and rising clear of neighbors without lateral collisions.

```
from pydobot import Dobot
from pydobot.dobot import MODE_PTP

device = Dobot(port='/dev/ttyUSB0')
device.speed(velocity=150.0, acceleration=150.0)

# Move above cube at hover height (Joint interpolation)
device.move_to(200.0, 0.0, 80.0, 0.0, mode=MODE_PTP.MOVJ_XYZ)

# Descend straight down to pick cube (Linear interpolation)
device.suck(True)
device.move_to(200.0, 0.0, 12.5, 0.0, mode=MODE_PTP.MOVL_XYZ)
```

Chapter 5: GUI Architecture with PyQt

5.1 Event Loop & Thread Decoupling

PyQt runs an internal **Event Loop** that processes user inputs and repaints the screen. Running ROS 2 subscription loops (`rospy.spin()`) on this main thread would cause the UI to freeze and stop responding.

The Solution: We isolate ROS 2 communication into a dedicated background worker thread, `RosBridge(QThread)`:

1. **Main UI Thread:** Listens for mouse clicks, manages layouts, and renders Qt widgets.
2. **Worker Thread ('RosBridge'):** Continuously calls `rospy.spin_once()` in the background. Incoming image frames and status JSONs are emitted across threads via thread-safe `pyqtSignal` instances, keeping the GUI 100% responsive.

Chapter 6: 3-Phase Mission Execution & Collision Avoidance

6.1 3-Phase Automated Mission Workflow

3-Phase Mission Architecture

1. **Phase 1 (Clear Obstacles):** Non-goal cubes located in the grid are picked and cleared to **Feeder Slots 1..4** on the left.
2. **Phase 2 (Stack Goal):** Target cubes are picked in sequence (#1..#4) and vertically stacked at **Center Goal [1, 1]** up to a strict 4-block limit ($Z = 30.0, 55.0, 80.0, 105.0$ mm).
3. **Phase 3 (Restore Obstacles):** Once goal stacking finishes, all obstacle cubes in the feeders are returned to their original grid slots.

6.2 Goal Collision Avoidance System

When 4 cubes are stacked at Goal [1, 1], the tower reaches 130 mm tall. A straight-line trajectory between a feeder ($Y = +79$ mm) and a right-column cell ($Y = -35$ mm) would slice directly through the center goal, toppling the tower.

Intelligent Detour & Clearance Trajectory:

- A cylindrical Keepout Zone ($R = 38.0$ mm) is established around Goal (176.0, 0.0).
- `_point_to_segment_dist()` evaluates candidate straight trajectories against this keepout cylinder.
- If a collision is detected, `_plan_safe_transit()` inserts a perimeter corridor detour waypoint:
 - **North Corridor Bypass:** Detours around the top perimeter via $X = 222.0$ mm.
 - **South Corridor Bypass:** Detours around the bottom perimeter via $X = 132.0$ mm.
- **Elevated Vertical Clearance:** Whenever cubes exist on the goal, transit elevation is automatically increased to $Z_{\text{transit}} = \max(\text{hover}_z, 142.0 \text{ mm})$, safely clearing the 130 mm tower.

Chapter 7: UI Dashboard Walkthrough with Screenshots

7.1 Stacking Mission Tab

Primary dashboard for configuring the 3-phase mission, assigning goal and obstacle roles, and executing autonomous stacking:

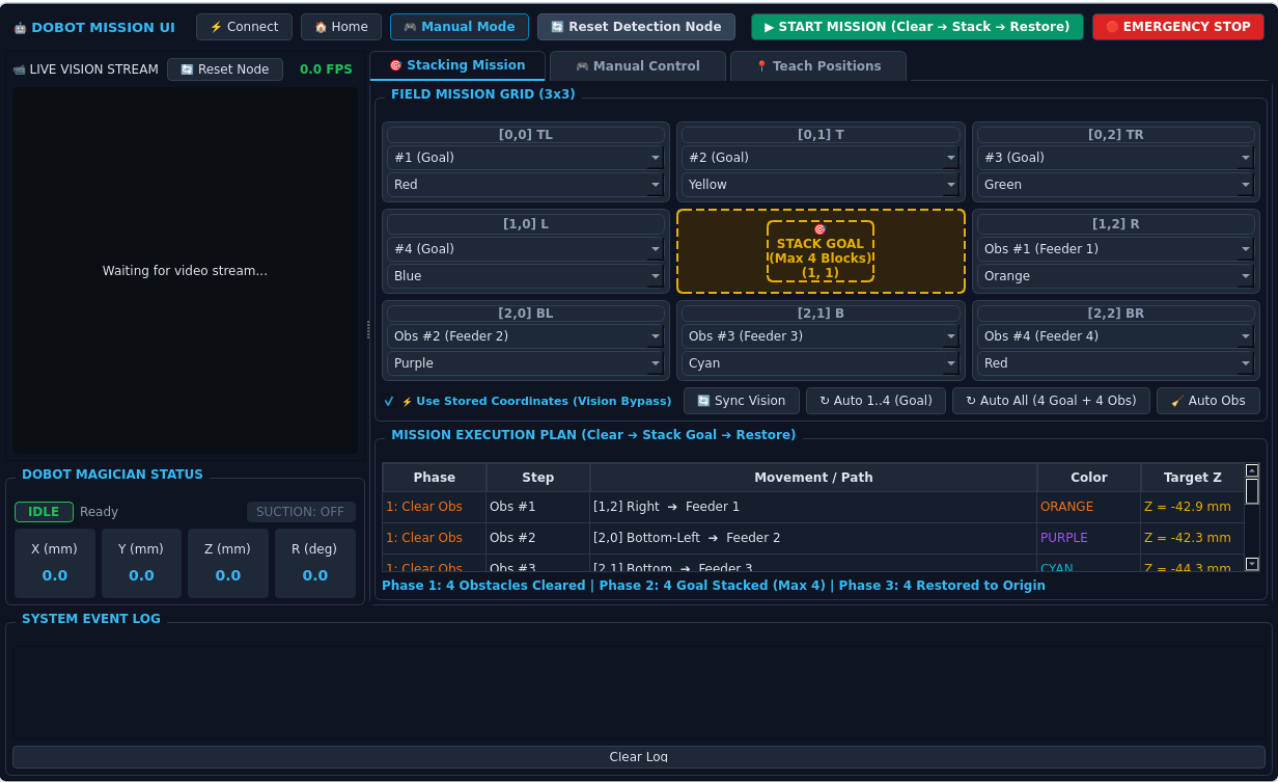


Figure 1: Stacking Mission Tab featuring live camera feed, 3x3 grid assignment cards with feeder routing, and 3-phase execution table

- **Auto 1..4 (Goal) Button:** Assigns orders #1 to #4 clockwise to Goal and clears remaining cells.
- **Auto All (4 Goal + 4 Obs) Button:** Assigns 4 Goal cubes (#1..#4) and routes remaining 4 cubes to Feeders 1..4.
- **Auto Obs Button:** Automatically routes unassigned colored cells to available feeder slots.
- **Execution Plan Table:** Displays color-coded task phases (Orange = Clear, Sky Blue = Stack, Teal = Restore).
- **START MISSION (Clear → Stack → Restore) Button:** Dispatches the mission to the controller node.

7.2 Manual Control Tab

Manual jogging and tool verification panel for setup and fine adjustments:

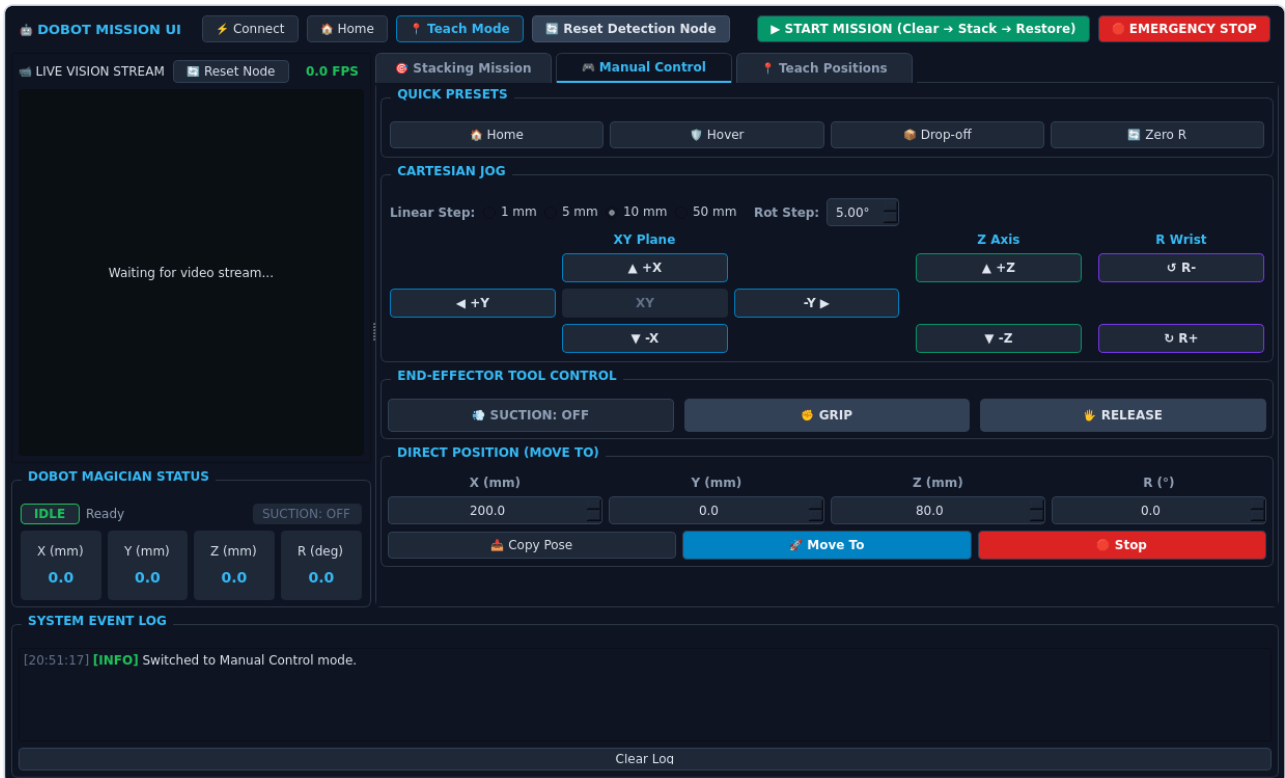


Figure 2: Manual Control Tab with Quick Presets, Cartesian Jogging D-Pad, Tool End-Effector toggles, and Direct Position inputs

- **Quick Presets:** Instant positioning to `Home`, `Hover` (Z=80mm), `Drop-off` (Z=30mm), or `Zero R`.
- **Cartesian Jog:** Directional buttons for X, Y, Z, R with selectable step sizes (1mm, 5mm, 10mm, 50mm).
- **Tool Control:** Dedicated toggles for suction cup pump (`SUCTION`) and gripper action (`GRIP` / `RELEASE`).
- **Direct Position:** Precision input fields for X, Y, Z, R coordinates with `Copy Pose` and `Move To`.

7.3 Teach Positions Tab (Vision Bypass)

Calibration interface for manually storing physical coordinates of each cell, the center goal, and feeders:

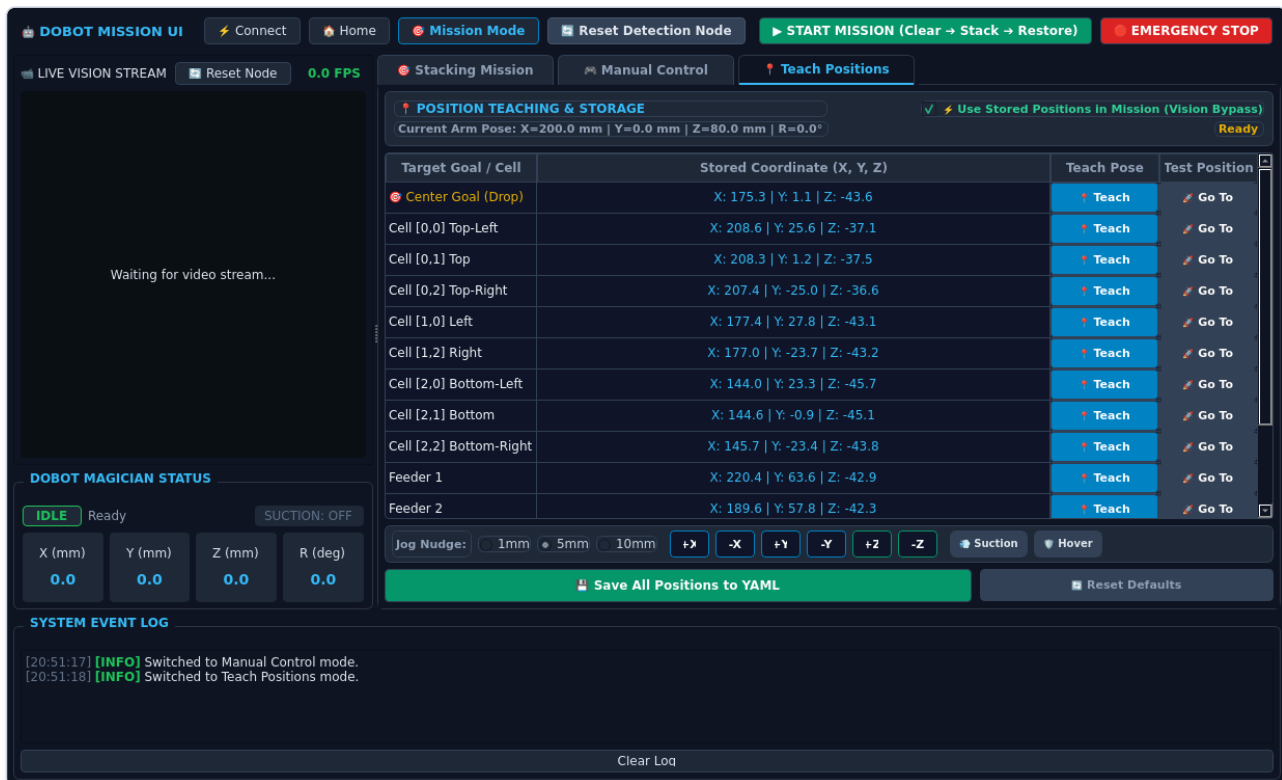


Figure 3: Teach Positions Tab for capturing and testing custom coordinates, saved directly to dobot_ui.yaml

- Jog the robot tip to touch the center of a desired cell or feeder.
- Click `Teach` in that cell's row to record live telemetry coordinates.
- Click `Go To` to verify repeatable positioning.
- Click `Save All Positions to YAML` to persist coordinates, enabling operation even if the camera is occluded or broken.

Chapter 8: Build, Execution & Troubleshooting Guide

8.1 Build & Launch Commands

```
# 1. Source ROS 2 Humble and build workspace
cd /home/thxncdzch/dobot_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash

# 2. Launch full system (Vision + Controller + GUI)
ros2 launch dobot_v2 dobot_system.launch.py

# Or launch GUI in Mock Simulation Mode (no physical robot required)
ros2 launch Dobot_UI dobot_ui.launch.py mock:=true
```

8.2 Troubleshooting & FAQ

⚠ USB Serial Permissions on Linux

If the controller reports Permission denied: '/dev/ttyUSB0':

```
sudo usermod -a -G dialout $USER
sudo chmod 666 /dev/ttyUSB0
# Log out and log back in for group changes to take effect
```

Camera Video Node Reset

If the video stream freezes or reports `Device Busy` , click the  **Reset Detection Node** button in the UI toolbar to cleanly restart the vision process in 0.5s.

Author: Thanadech (Google Deepmind Pair-Programming Assistant - Antigravity) • September 2026